# Concurrency at Scale

Some slides adapted from:
Chris Rossbach CS378h at UT Austin
Kishore Pusukuri CS5412 at Cornell

# Administrivia

Quiz at EOD

Poll on final lecture topics on Ed

# Agenda

- Executing dependency graphs on one machines (hw8, hw9)
- Execution dependency graphs on multiple machines (basis for next week)

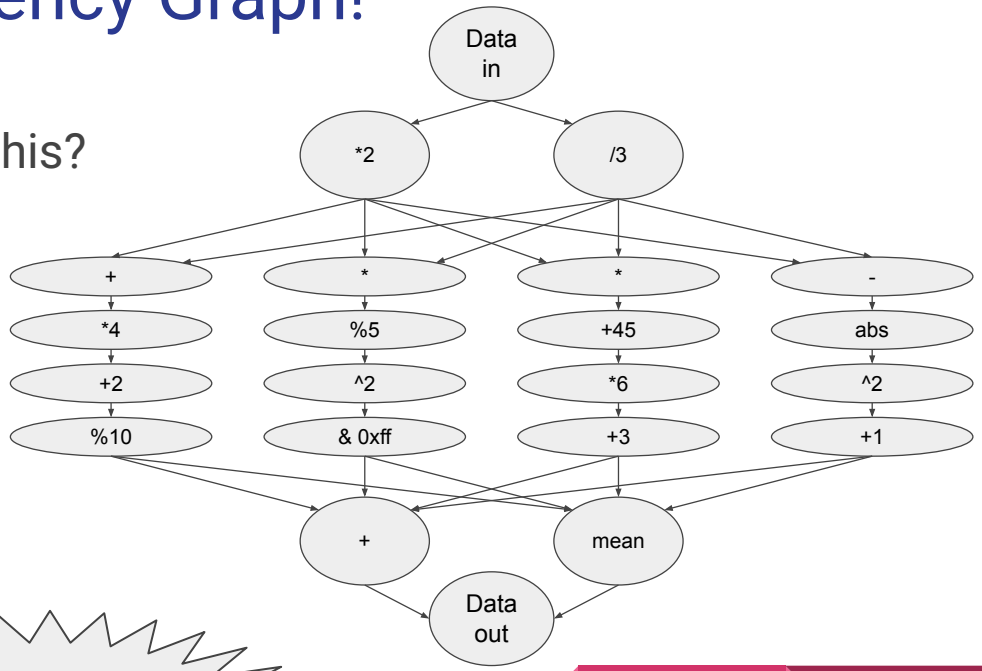
# How do we execute some concurrent operations?

- Input data
- Workers (CPUs or threads)
- Tasks (via functional & domain decomposition)

# Goal: Execute this Dependency Graph!

What are the ways we can decompose this?

- Functional decomposition
  - 1 node = 1 task
- Domain decomposition
  - 1 data = 1 task
- Both
  - 1 data in 1 node = 1 task

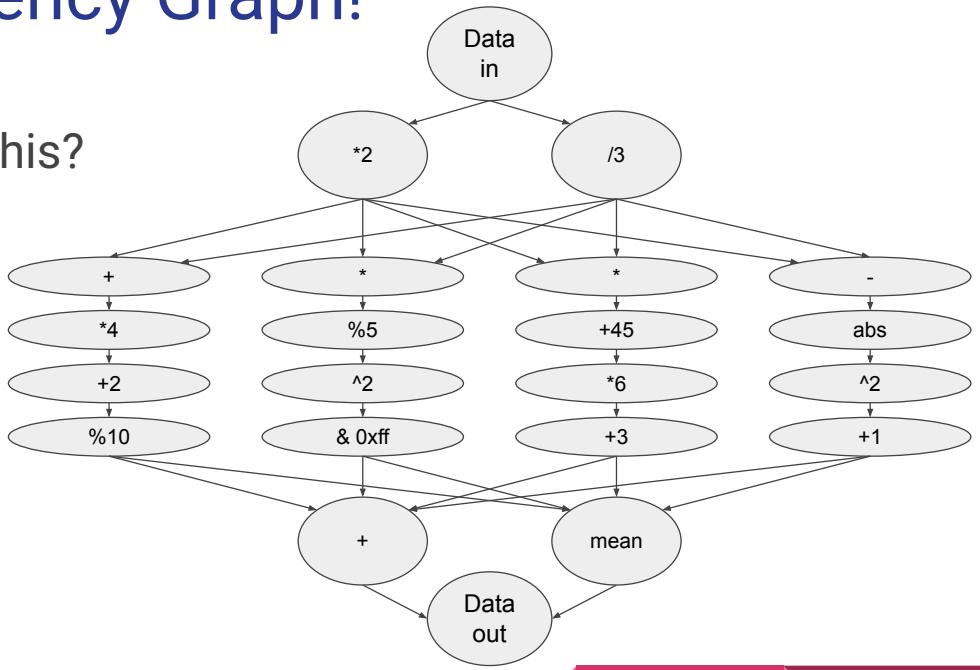



Which is best!?

# Goal: Execute this Dependency Graph!

What are the ways we can decompose this?

- What if one downstream task becomes a bottleneck?
- Work might not be embarrassingly parallel across data
- Large # of thread creations



# How do we implement this?

```
void main() {
    for (d in data) {
        for (node in graph) {
            chans = create_channels();
            thread_spawn(node, chans);
            send(chans[0], d);
}}}
```

Doesn’t work well for small data & graphs

Doesn’t work at all for huge data

# Using the F&D decomposition…

- Dataset of 10M elements times 100 nodes in the graph => bad news…
  - Each thread takes at least 8MB of memory for its stack => 8 Petabytes…
- 

```
$ cat /proc/sys/kernel/threads-max
510944
```

Does creating threads dynamically help?

# New idea…

Each thread spawns its children only once they can run, e.g.

```
void start(Data d) {
    h1 = spawn(+2)
    h2 = spawn(/3)
    h1.join(); h2.join();
    h3, h4, h5, h6 = spawn(the four next children, which
spawn their children…)
}
```

# Implicit Dependency Graph

- Graph is constructed dynamically
  - Lower performance
  - Even more flexibility
- Perhaps we can find a way to not overwhelm with a large number of threads by slowly spawning them

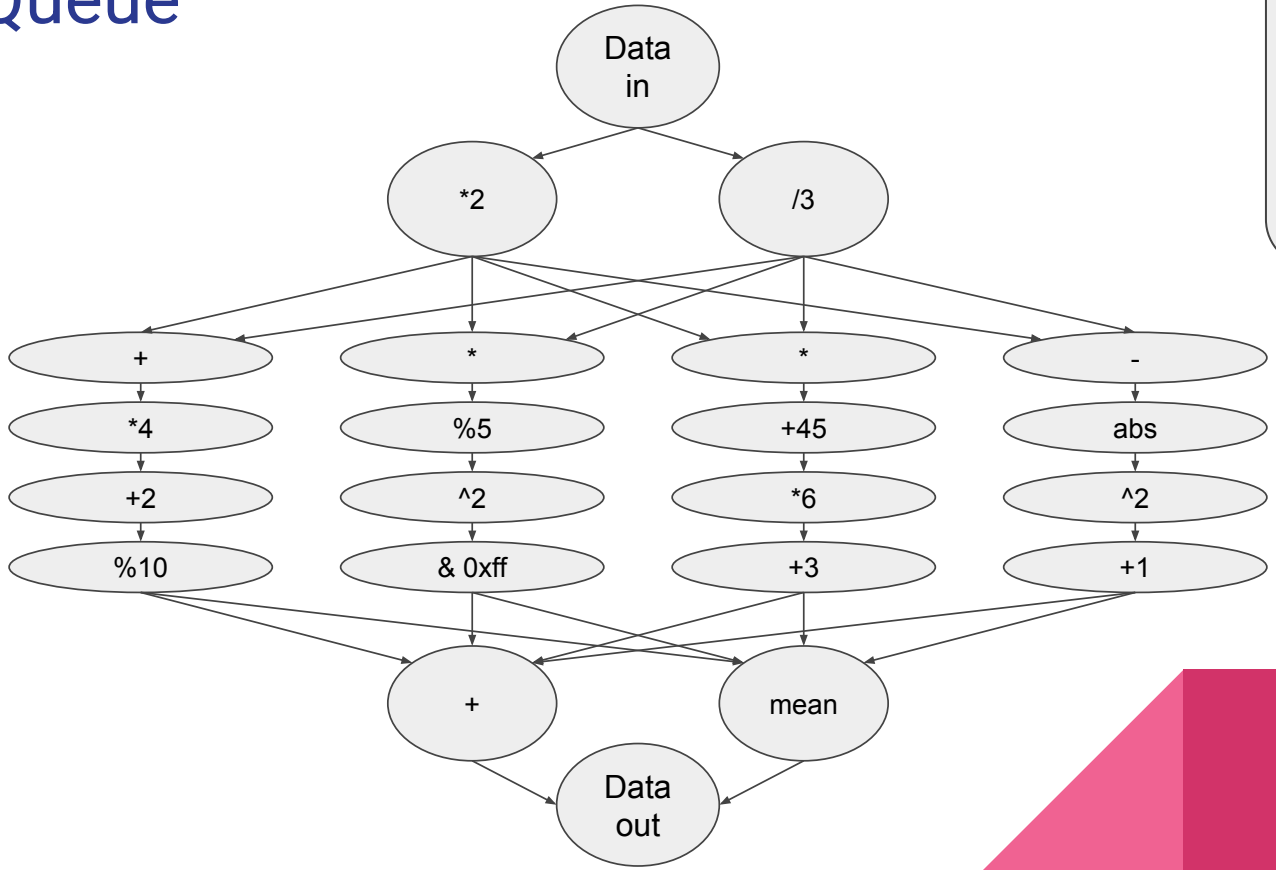
# A new problem exists

- Intermediate data blows up even if the number of threads is kept reasonable!

# Ready Queue

Rdy Queue

- Data[0]*2
- Data[0]/3
- Data[1]*2
- Data[1]/3
- ...




Done

# Ready Queue

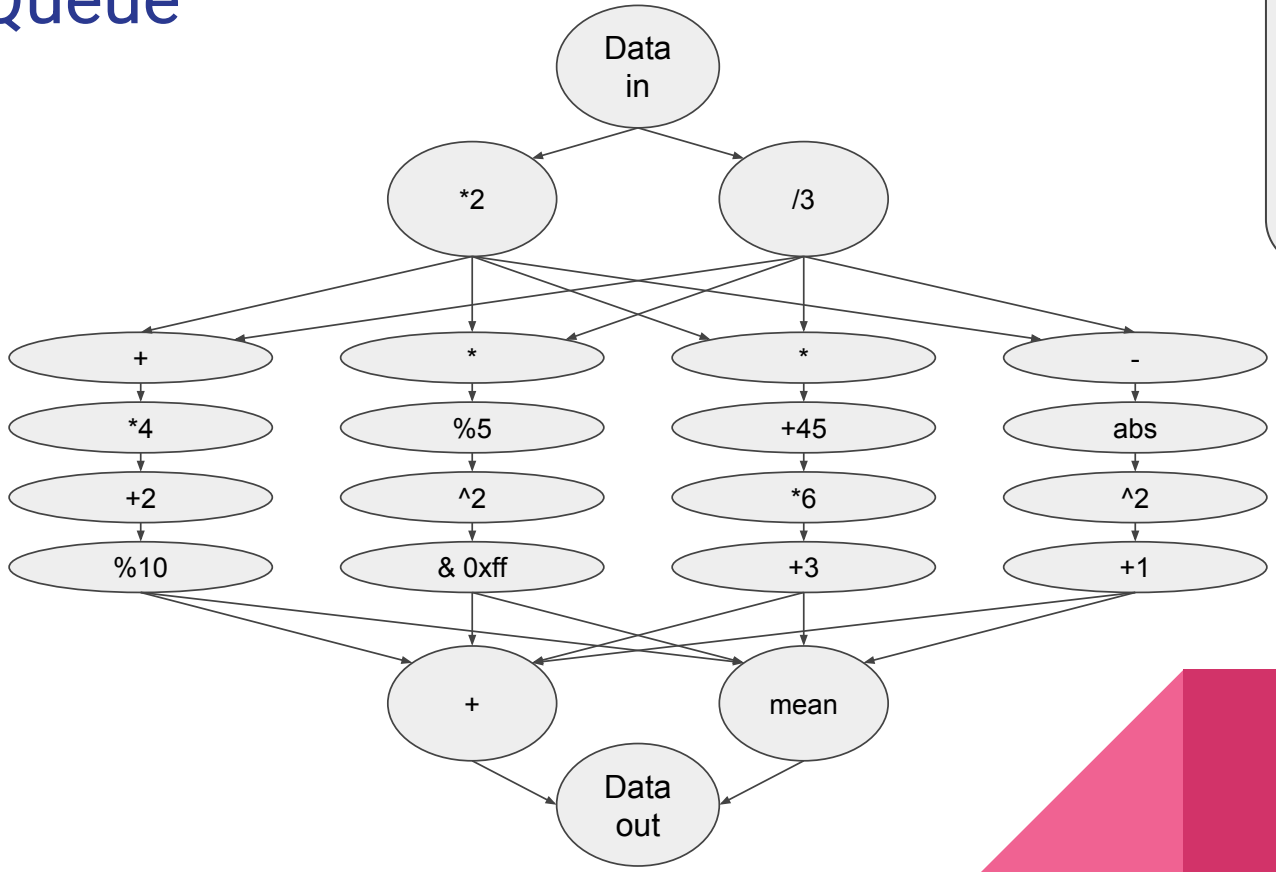
Rdy Queue

Data[0]*2

Data[1]*2

Data[1]/3

...



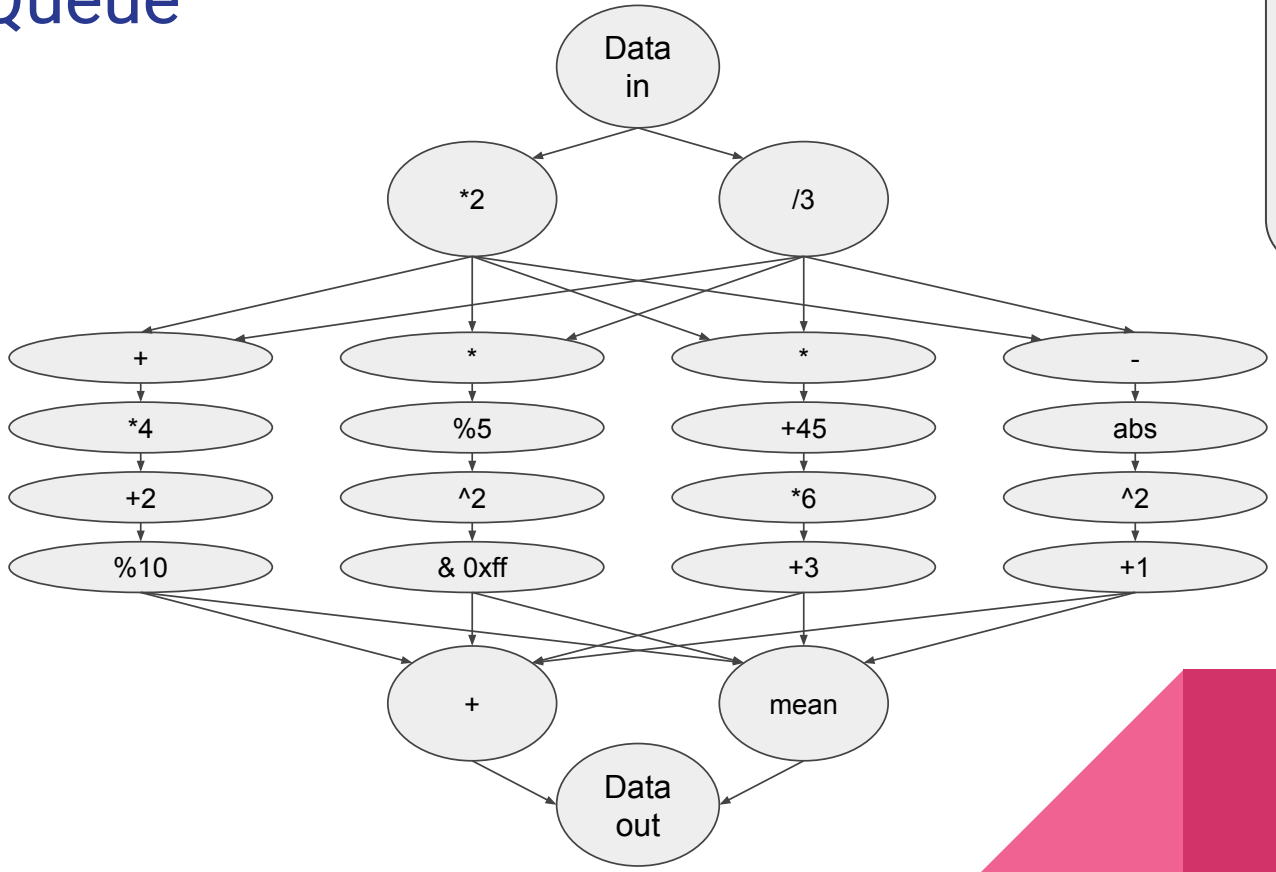

Done

Data[0]/3

# Ready Queue

Rdy Queue

- Data[1]*2
- Data[1]/3
- ...
- L[0]+L[1]
- L[0]*L[1]
- L[0]*L[1]
- L[0]-L[1]




Done

- Data[0]*2
- Data[0]/3

# Takeaways

- We need a fixed number of threads
  - But we still want a dynamic amount of work to exploit as much parallelism as possible!
- We need tasks to run to completion
  - To avoid intermediate data blowup
- What to do? Create a new abstraction
  - Spawn 100 threads, have them pull work off of a work queue

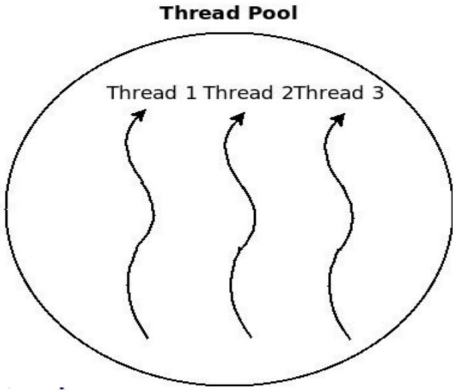
# Thread Pools

A popular backend for executing dependency graphs

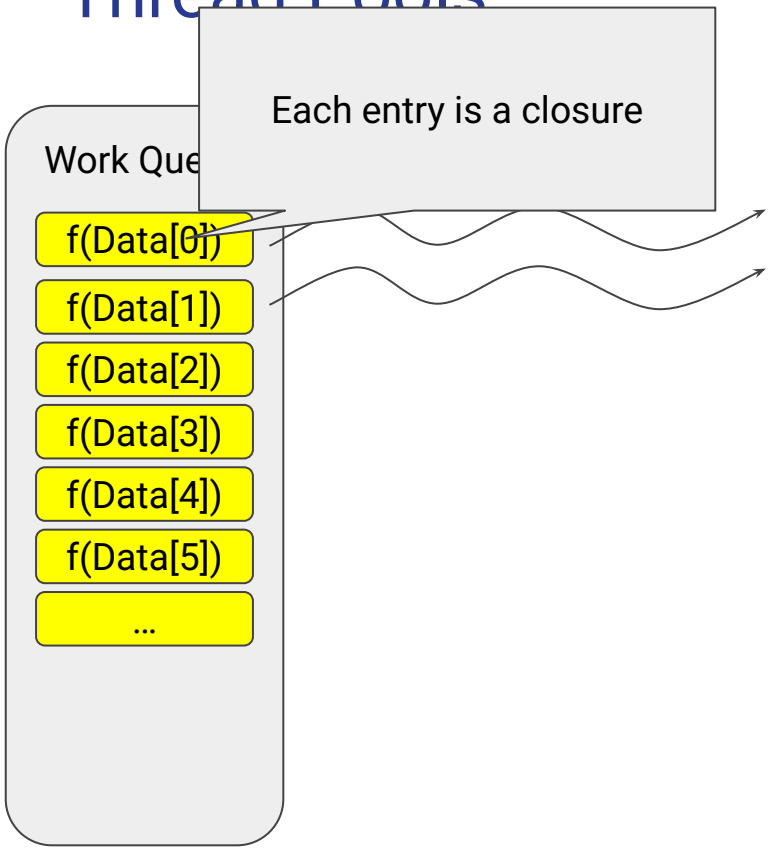
Spawn a fixed number of threads, let the caller handle dependencies

Thread loop: while work exists, run the work. (what is “work” now?)

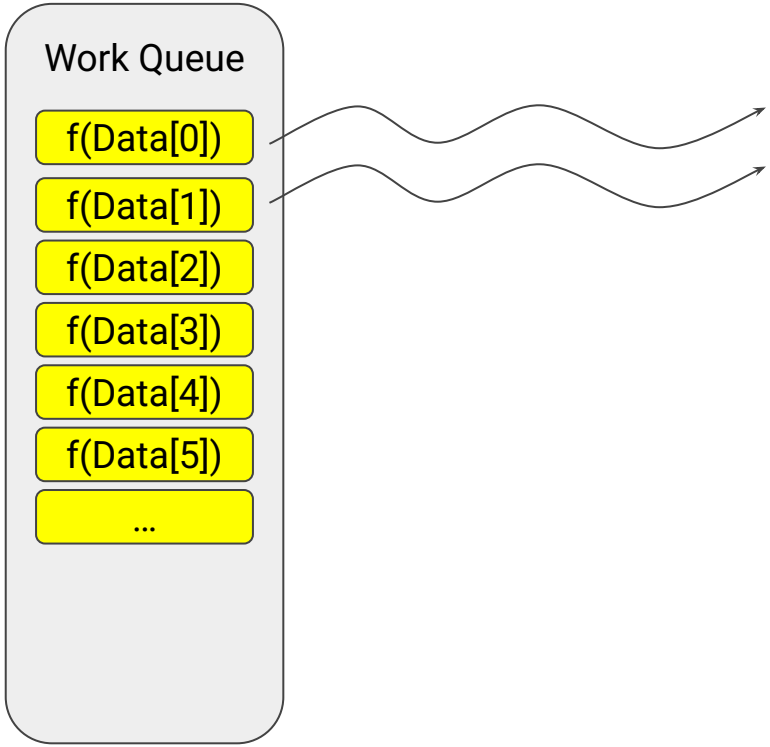
- Key: ~32 bytes per item in the ready queue scales much better than 8MB/item



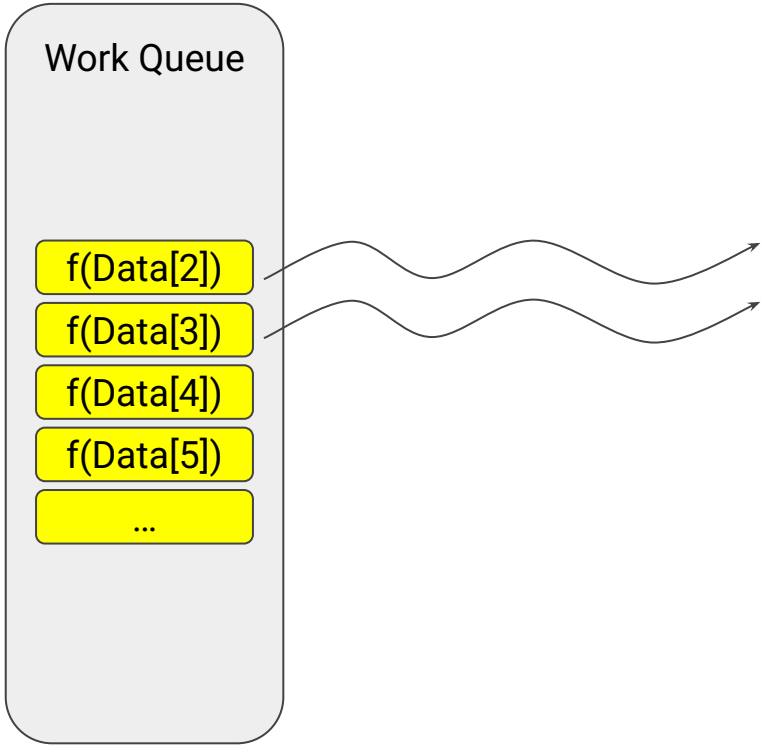
# Thread Pools

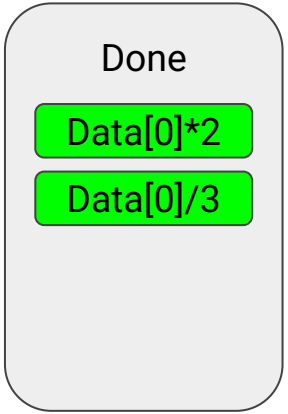


# Thread Pools

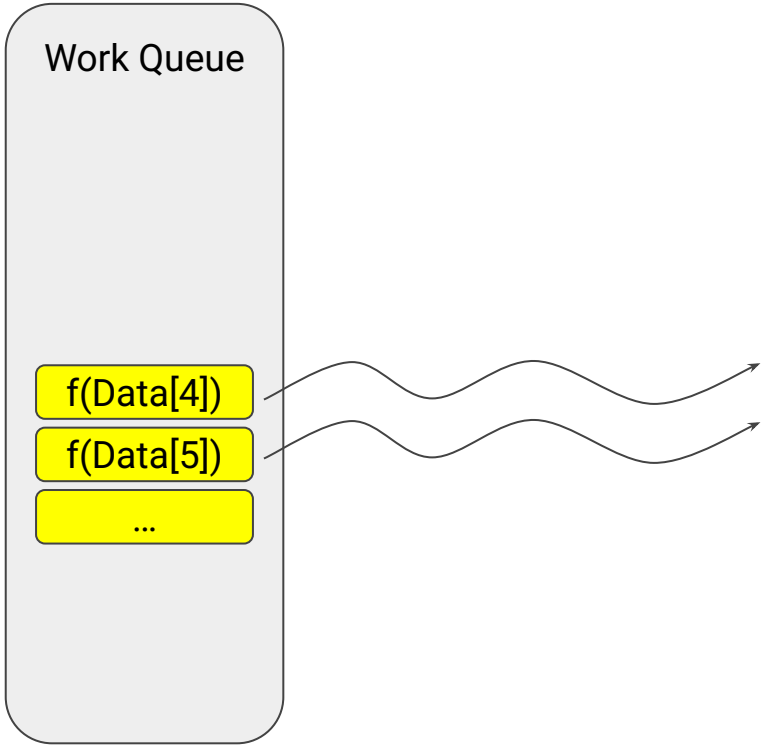


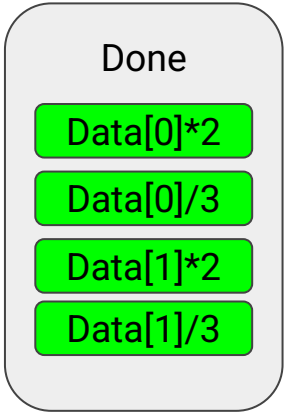
# Thread Pools



# Thread Pools



# Thread Pools

Insight: no point having many more threads than N_CORES!

Work is something we can represent much more compactly than a thread

Downsides: we cannot context switch within a work! Work is run to completion.

# Thread Pool API

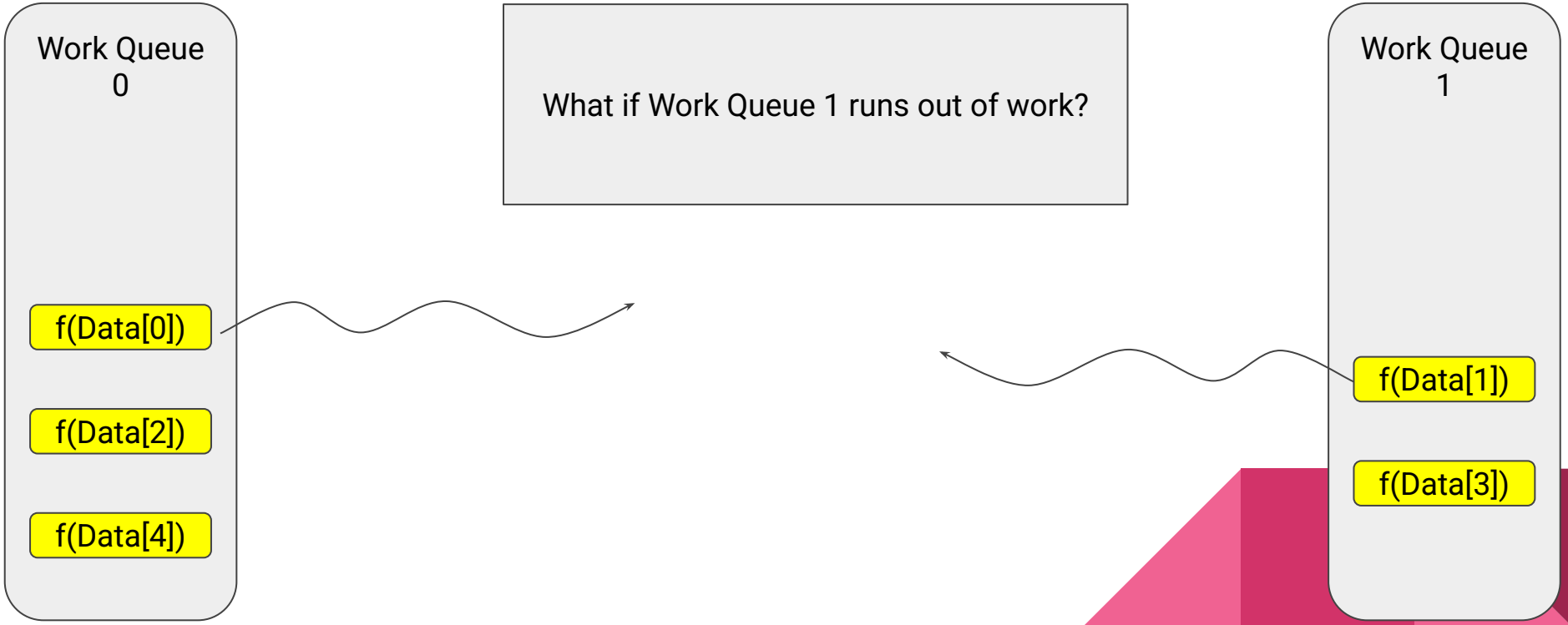
```
handle = thread_pool.submit(work)

is_done_no_block = handle.status()

result = handle.join()
```

# Thread Pools: Contention

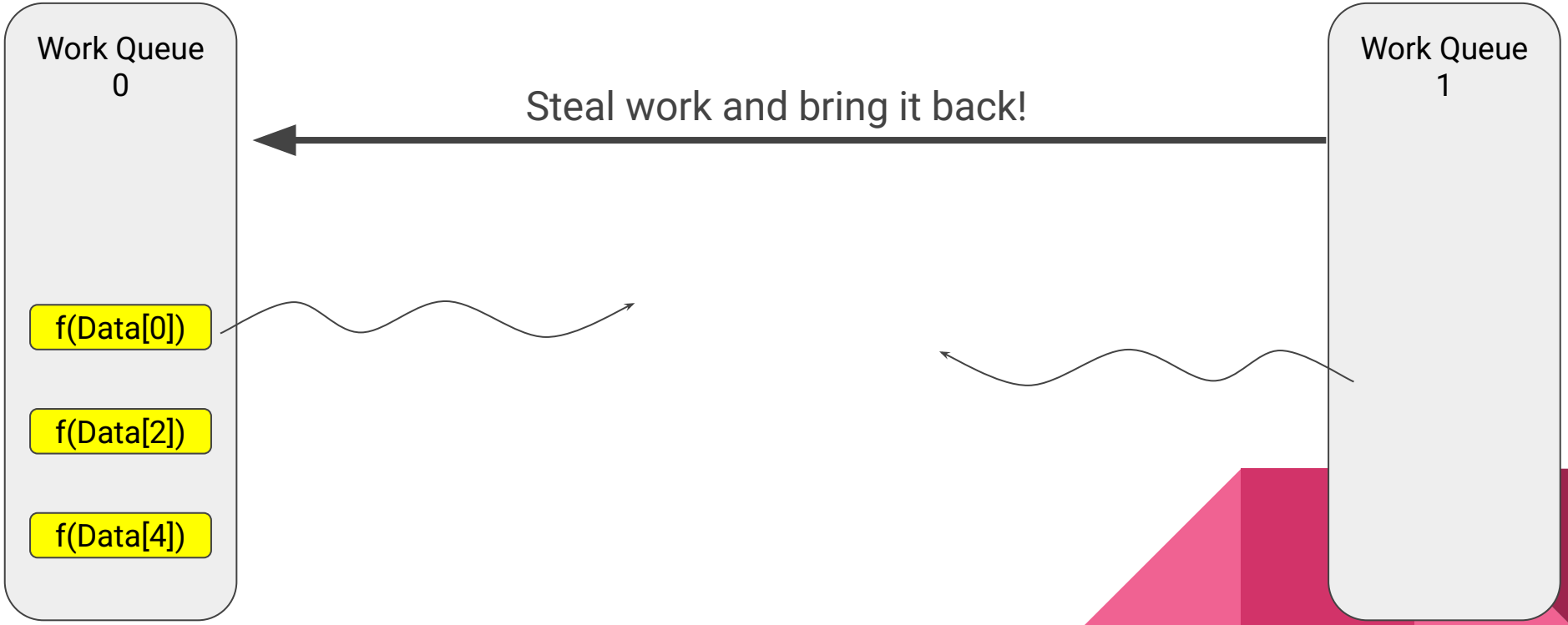
Each thread in the pool must access the same work queue…

What if each thread had its own work queue?

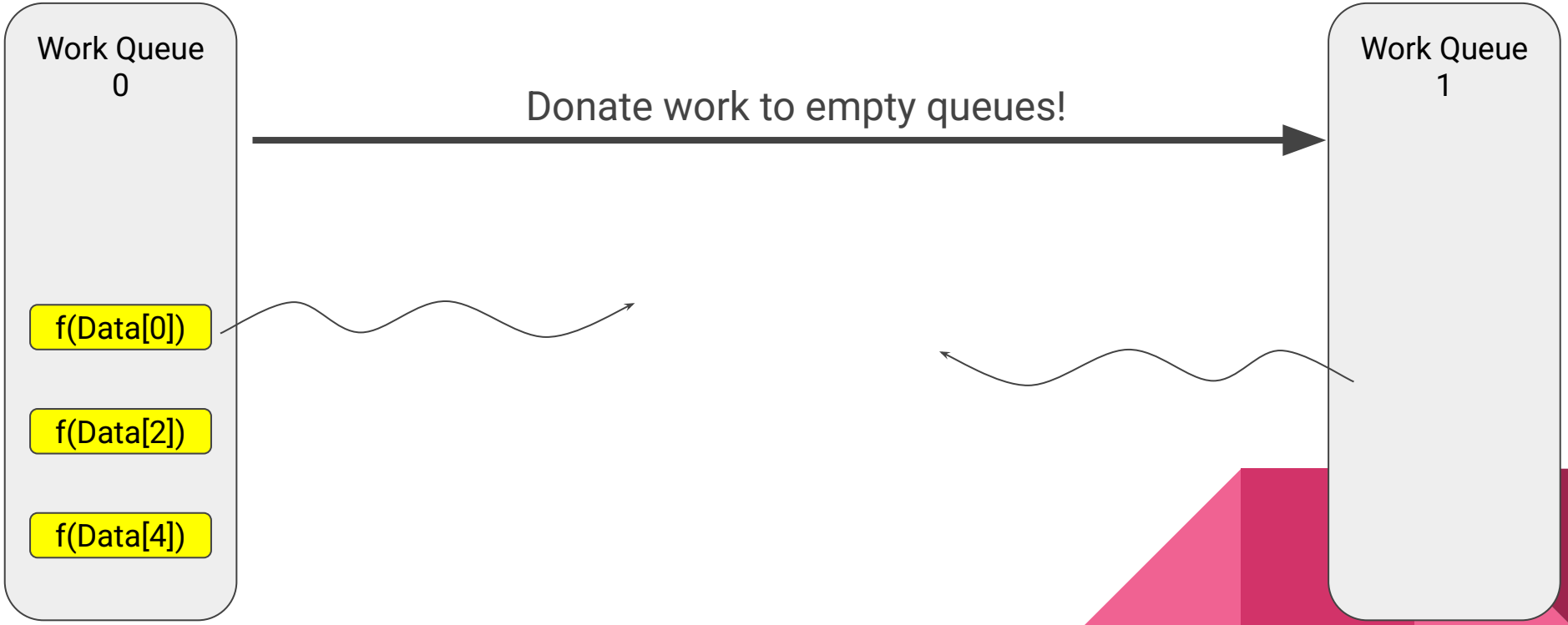
# Thread Pools



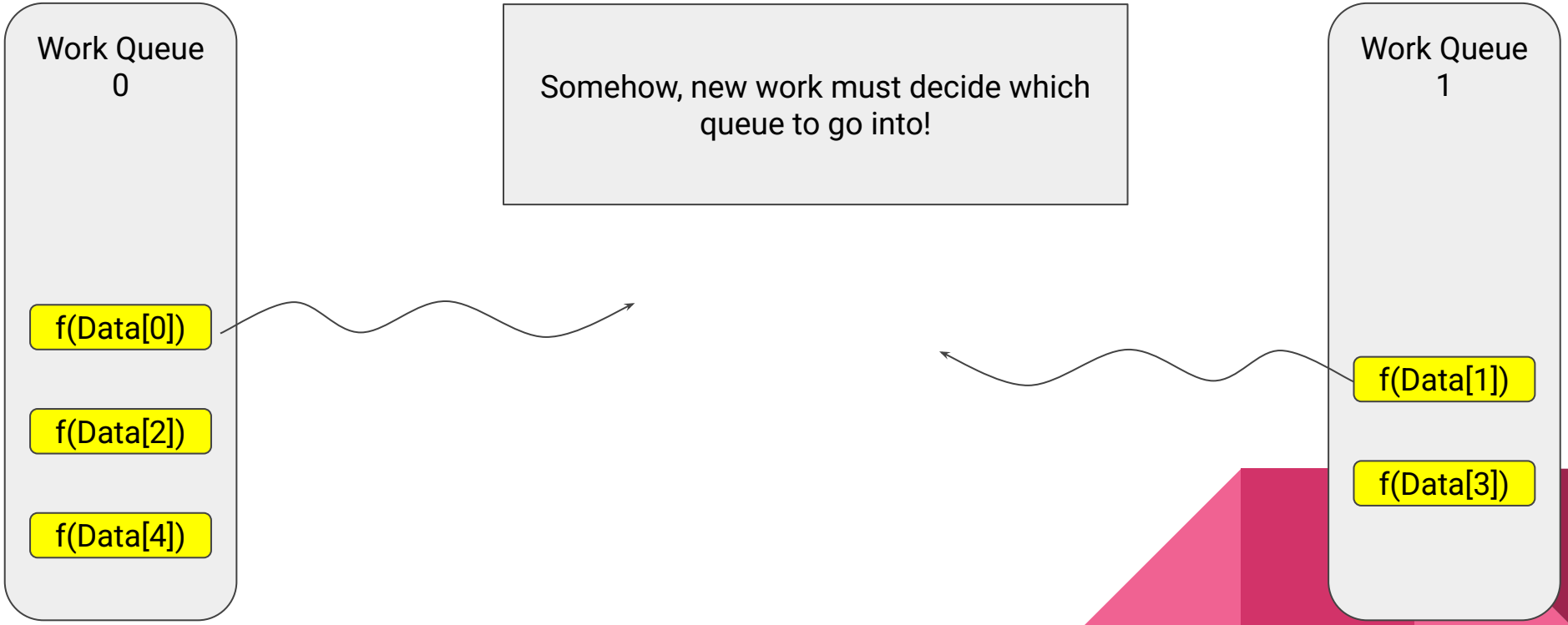
# Thread Pools: Work Stealing



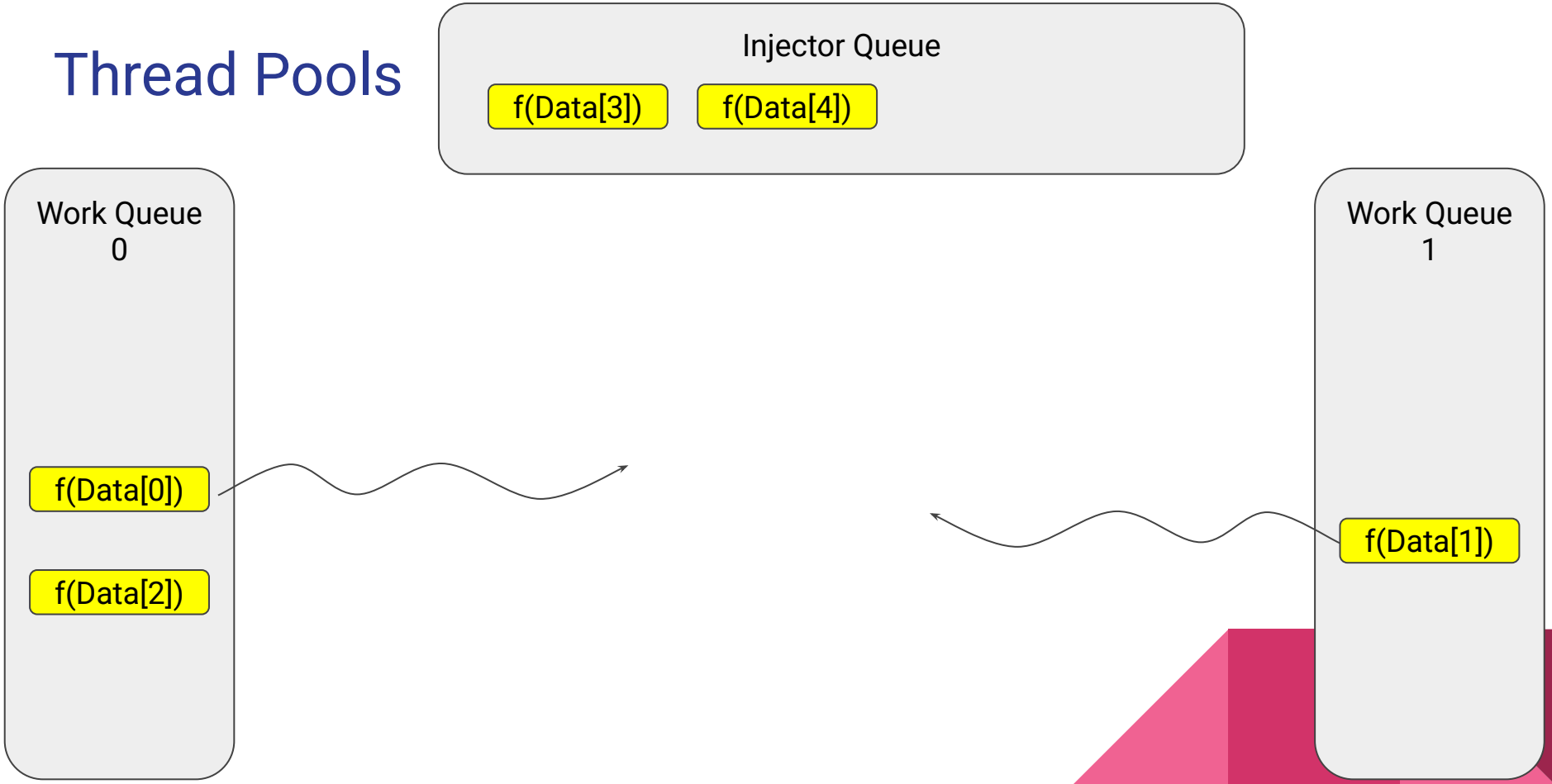
# Thread Pools: Work Donating



# Thread Pools



# Thread Pools



# Wait, aren't we just reimplementing a scheduler?

CPUs run threads

Expensive to make & switch between

going into the kernel takes a bit

threads run work

Cheap to make & switch between

Every problem in Systems can be solved with just 1 more layer of indirection

# Thread Pool analysis

- CPU-bound tasks
  - Works great
- I/O-bound tasks
  - Works terribly
  - Underlying threads will block on the I/O, and not run other work…
    - Event based programming (next time) remedies this!

# Rayon

Rust’s popular thread pool frontend

# Rayon Usage

```
fn sum_of_squares(input: &[i32]) -> i32 {
    input.par_iter() // <-- just change that!
         .map(|&i| i * i)
         .sum()
}
```

# Agenda

- ~~Executing dependency graphs on one machines~~
- Execution dependency graphs on multiple machines

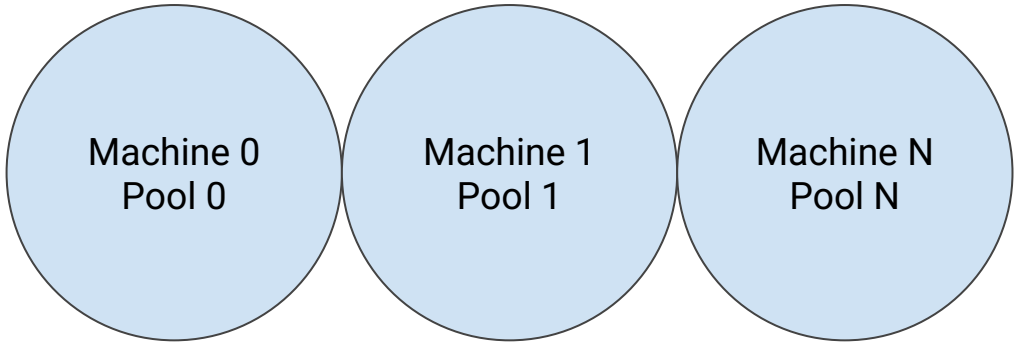
# Now we have an efficient Dependency executor!

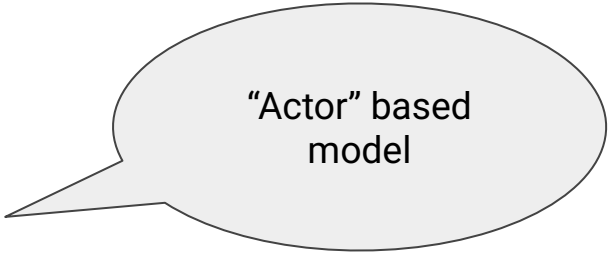
New goal: make it scale to multiple machines

No shared memory
Machines can fail
High communication cost

# How are we going to do this?

1 approach: each machine runs a thread pool, send and receive between machines to coordinate work



# Distributed Thread Pools

- Pros
  - Good for embarrassingly parallel / low dependency workloads with no communication
  - E.g. distributed supercomputing, simulations, bitcoin mining
- Cons
  - No shared memory -> moving lots of intermediate data around!
  - Programming is extremely low-level (have to worry about each individual send/recv to build up to a sum)
  - Synchronization is hard, so reductions are hard (e.g. sum(data))
    - Think parallel prefix sum (hw2), now each barrier needs to be cross-machine and send all new updates to all other machines

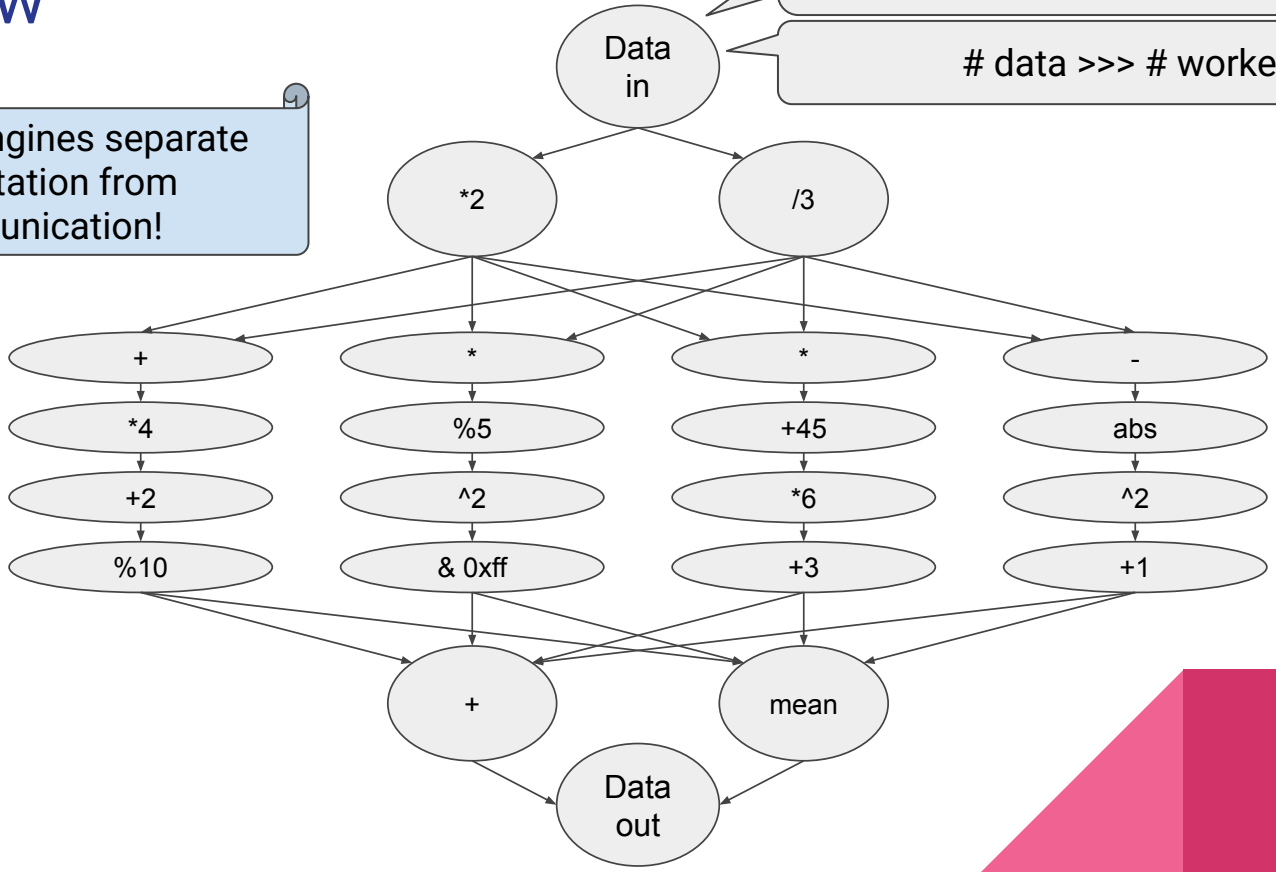
We need a different abstraction to handle more complex programs at scale!

# Dataflow

Dataflow engines separate computation from communication!

Define the graph statically instead of dynamically!

# data >>> # workers



# Dataflow Engines

Programming model + infrastructure (runtime)

Write programs that run on lots of cores on lots of machines

Automatic parallelization and distribution

I/O and jobs scheduling

Status and monitoring

Fault-tolerance

All modern “big data” platforms are **dataflow engines**!

Differences:

- What graph structures are allowed?
- How does this impact programming model?

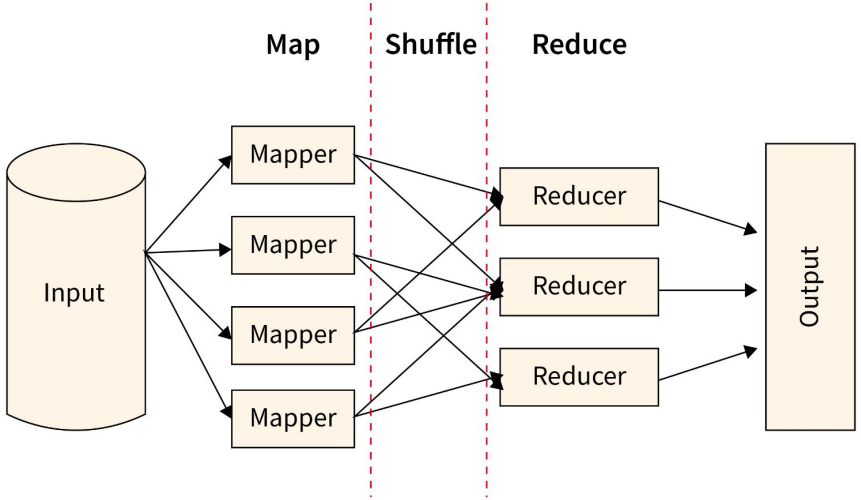
# Dataflow Engine

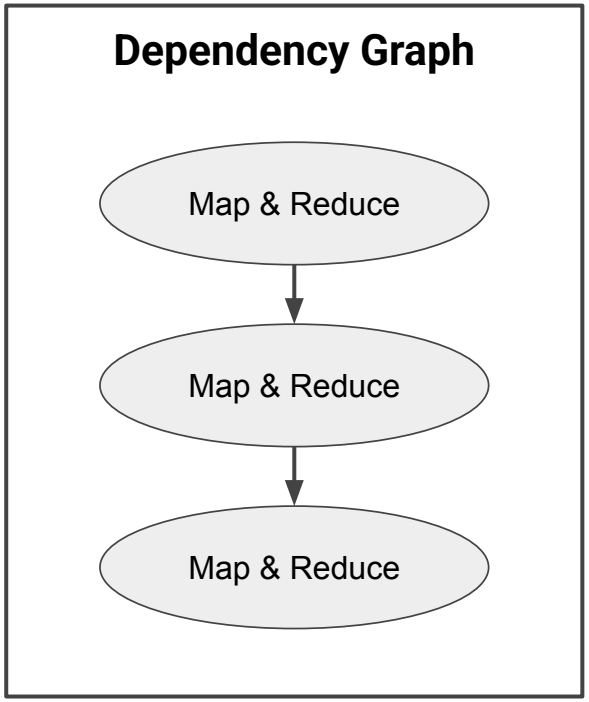
3 pieces:

- Programming model
  - Provides abstractions that force users to write programs that will scale well
- Coordinator
  - Tries to intelligently schedule such that intermediate data doesn’t have to move across machines much
- Fleet
  - Just runs work given by coordinator (via a local thread pool)

# MapReduce

- One of the earliest “dataflow” engines
- Only one node type: a combined map and reduce
- Tasks are generated from MRs



# Map (N to N)

Apply function to every element of a dataset

# Map

f better be a “pure” function (no side effects) (Fn in Rust)

```
fn map<T, U>(f: fn(T) -> U, d: Vec<T>) -> Vec<U> {
    let mut r = Vec::<U>::new();
    for i in d {
        r.push(f(i));
    }
    r
}
```

# Reduce (N to 1)

Combine two items of a dataset via reduce(x1, x2)

Recurse in tree pattern until you end with one final value

# Reduce

```
fn reduce<T>(f: fn(T, T) -> T, d: Vec<T>) -> T {
    if d.len() == 1: d[0]
    f(d[0], reduce(f, d[1..]))
}
```

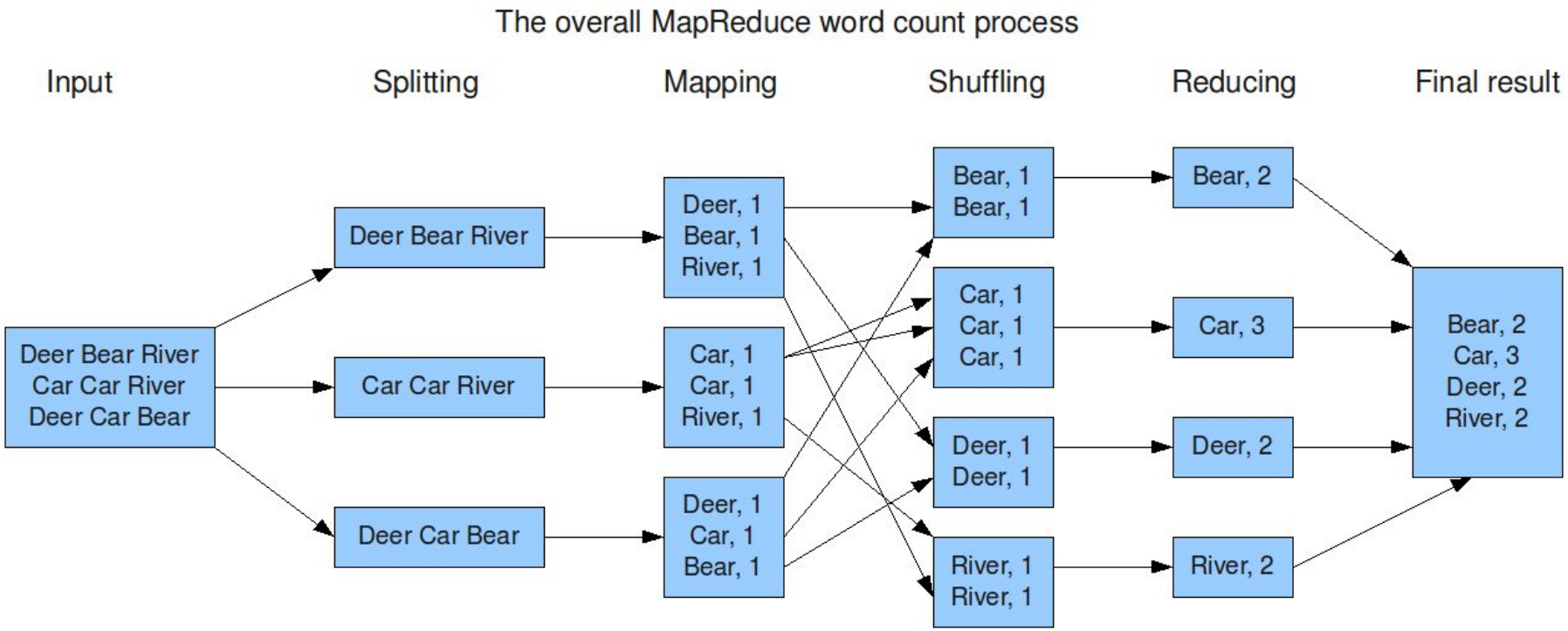
Prefix sum?

# ReduceByKey

Reduce over a HashMap (K, V) instead of a Vec (T)

Only reduce items with identical Keys

- Apply reduce to each key’s bucket


The overall MapReduce word count process
Input
Splitting
Mapping
Shuffling
Reducing
Final result
Deer Bear River
Car Car River
Deer Car Bear
Deer Bear River
Car Car River
Deer Car Bear
Deer, 1
Bear, 1
River, 1
Car, 1
Car, 1
River, 1
Deer, 1
Car, 1
Bear, 1
Bear, 1
Bear, 1
Car, 1
Car, 1
Car, 1
Deer, 1
Deer, 1
River, 1
River, 1
Bear, 2
Car, 3
Deer, 2
River, 2
Bear, 2
Car, 3
Deer, 2
River, 2

# MapReduce Criticism

## MapReduce: A major step backwards | The Database Column

http://databasecolumn.vertica.com/database-innovation/mapreduce-a-major-step-backwards/ September 6, 2011

on Jan 17 in Database architecture, Database history, Database innovation posted by DeWitt

*[Note: Although the system attributes this post to a single author, it was written by David J. DeWitt and Michael Stonebraker]*

On January 8, a Database Column reader asked for our views on new distributed database research efforts, and we'll begin here with our views on MapReduce. This is a good time to discuss it, since the recent trade press has been filled with news of the revolution of so-called "cloud computing." This paradigm entails harnessing large numbers of (low-end) processors working in parallel to solve a computing problem. In effect, this suggests constructing a data center by lining up a large number of "jelly beans" rather than utilizing a much smaller number of high-end servers.

For example, IBM and Google have announced plans to make a 1,000 processor cluster available to a few select universities to teach students how to program such clusters using a software tool called MapReduce [1]. Berkeley has gone so far as to plan on teaching their freshman how to program using the MapReduce framework.

As both educators and researchers, we are amazed at the hype that the MapReduce proponents have spread about how it represents a paradigm shift in the development of scalable, data-intensive applications. MapReduce may be a good idea for writing certain types of general-purpose computations, but to the database community, it is:

1. A giant step backward in the programming paradigm for large-scale data intensive applications

# MapReduce Criticism

- Dependency graph is limited
  - Ignores decades of DB research
  - Useful operations like InnerJoin, OuterJoin of datasets are inefficient to program
- Persistence is applied between every MR
  - No way to customize which data is “intermediate”
  - Write and re-read full dataset every iteration!

# Apache Spark

The modern dataflow engine

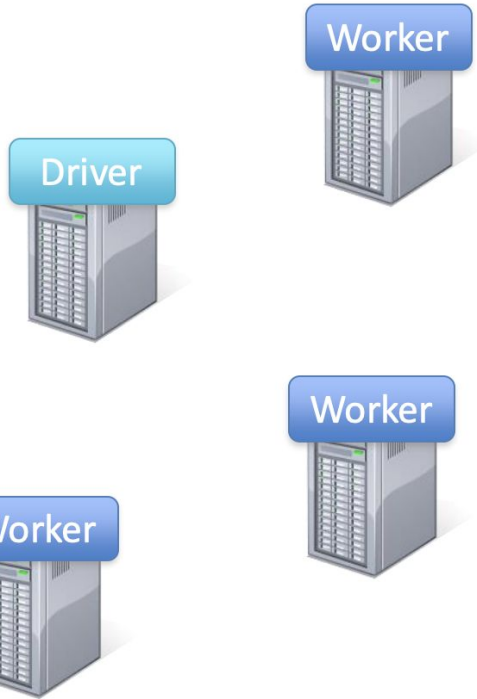
Customize your dataflow graph by writing a program

- Allows for DAGs, not just the MR BSP model

“Resilient Distributed Dataset” is their primitive data type

# Spark Program: Log Mining

```
lines = spark.textFile("hdfs://...")
errors = lines.filter(_.startsWith("ERROR"))
messages = errors.map(_.split('\t')(2))

cachedMsgs = messages.cache()
cachedMsgs.filter(_.contains("foo")).count
cachedMsgs.filter(_.contains("bar")).count
```




Result: full-text search of Wikipedia in 0.5 seconds vs 20

# Resilient Distributed Dataset

RDDs must be immutable to do this!

- Resilient
  - Each output data keeps track of its *lineage*
  - If a node fails, you can recover the lost data!
- Distributed
  - Each RDD is composed of multiple *partitions*
  - Each partition is processed on a different node in the cluster
- Dataset
  - Initial data can come from a file or be transformed from an RDD

# RDD Operations

1. Transformations (define new RDD from old)
   a. Map, filter, groupByKey, sortByKey
   b. Lazy
2. Actions (Get a single value from an RDD)
   a. Count, collect, take (collect top N), saveToTextFile
   b. Drive computation to occur

“Pull model” - actions trigger execution

# RDD Example 2

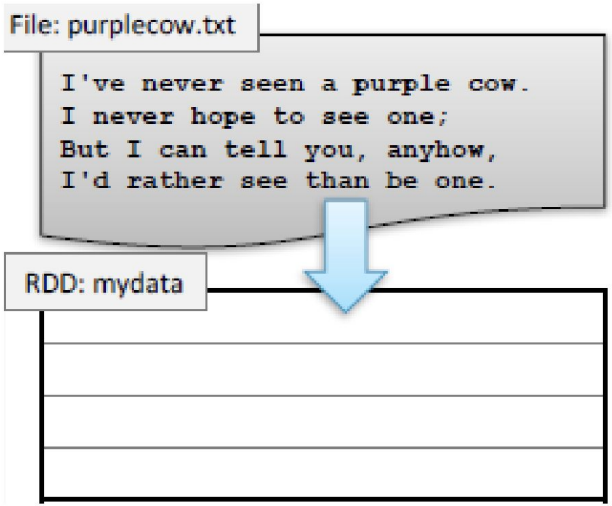
File: purplecow.txt

```
I've never seen a purple cow.
I never hope to see one;
But I can tell you, anyhow,
I'd rather see than be one.
```

```
>
```

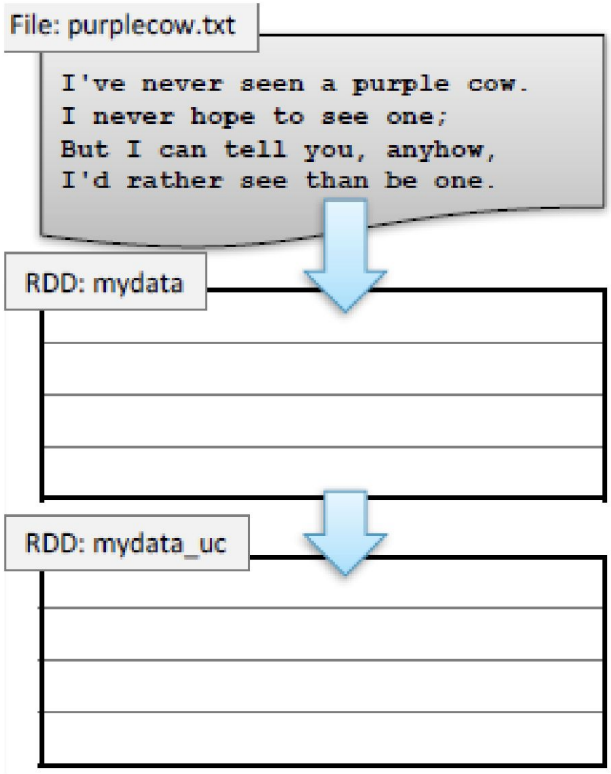
# RDD Example 2

```
> mydata = sc.textFile("purplecow.txt")
```



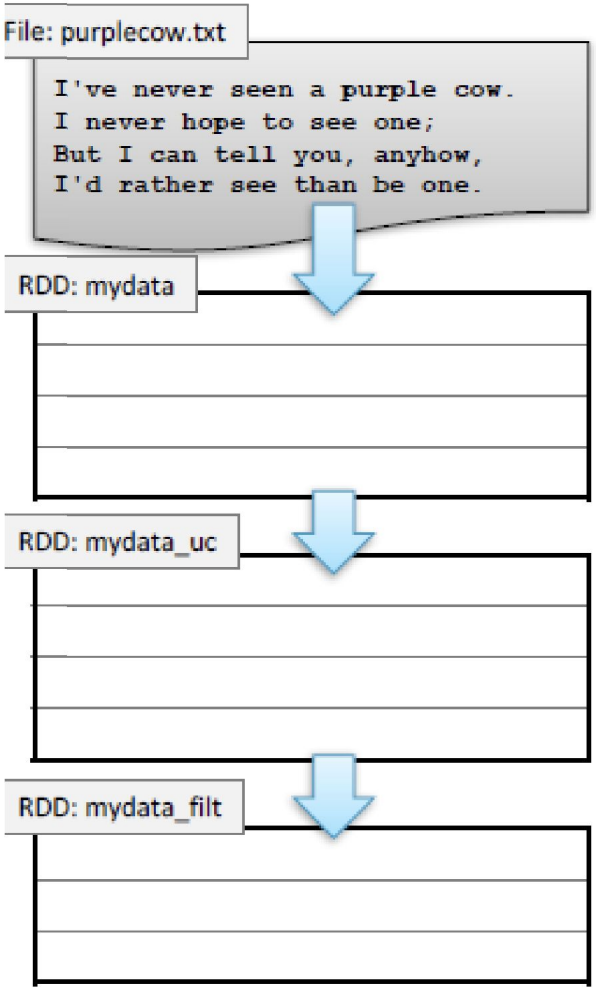
# RDD Example 2

```
> mydata = sc.textFile("purplecow.txt")
> mydata_uc = mydata.map(line =>
line.toUpperCase())
```



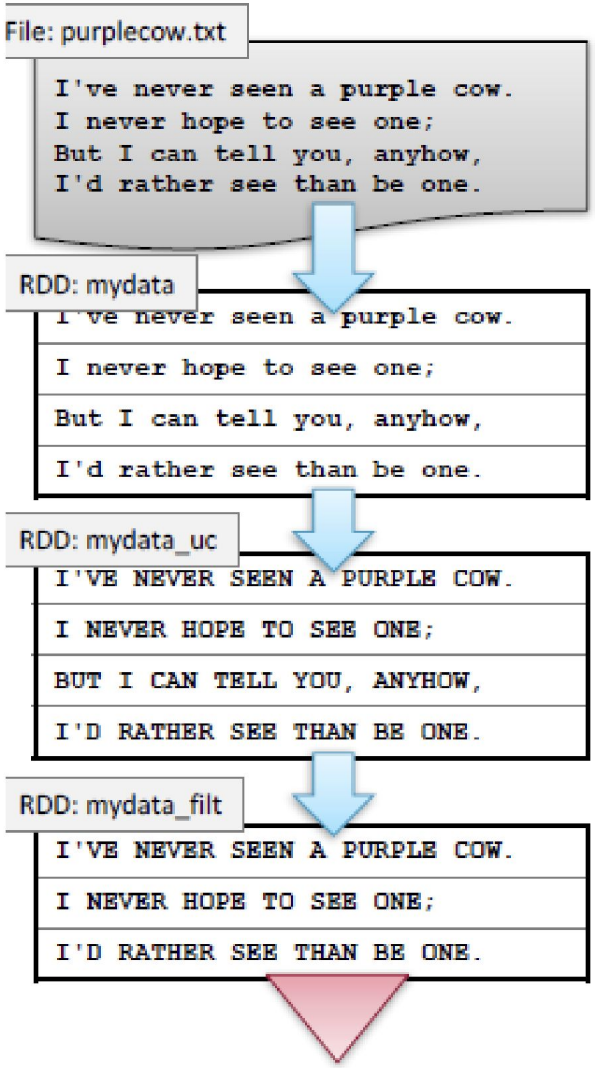
# RDD Example 2

```
> mydata = sc.textFile("purplecow.txt")
> mydata_uc = mydata.map(line =>
line.toUpperCase())

> mydata_filt = mydata_uc.filter(line =>
line.startsWith("I"))
```

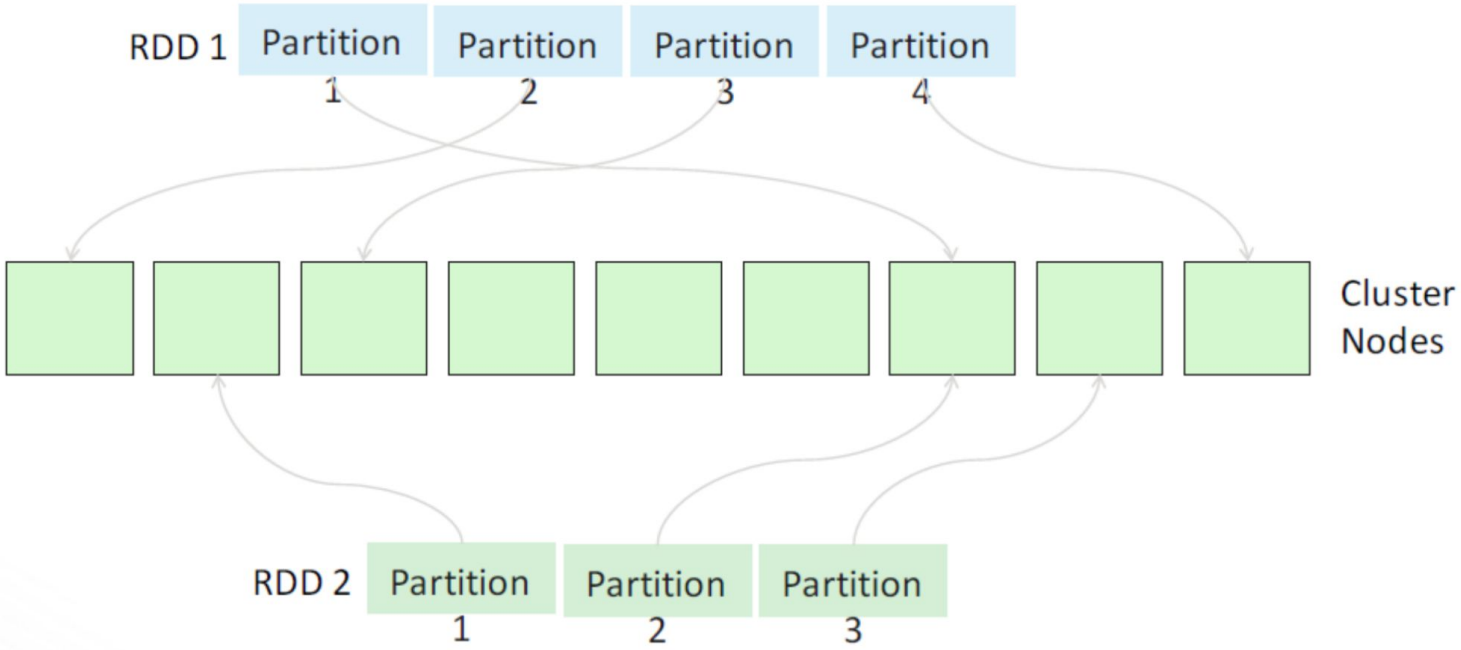


# RDD Example 2

```
> mydata = sc.textFile("purplecow.txt")
> mydata_uc = mydata.map(line =>
line.toUpperCase())

> mydata_filt = mydata_uc.filter(line =>
line.startsWith("I"))

> mydata_filt.count()
3
```



# Spark Partitioning



# Spark Key Ideas

- RDD Operations are designed to offer embarrassing parallelism
  - Limits programmability, all nodes do ~the same thing
- Spark intelligently maintains partitions of data to maximize local computation
  - Minimize data communication
- If some component is running behind, Spark launches a substitute

# TensorFlow

```
inputs = tf.keras.layers.Input(shape=(42,))
x = tf.keras.layers.Embedding(input_dim=20000,
                              output_dimension=50,
                              input_length=42,
                              mask_zero=True)(inputs)
x = tf.keras.layers.Flatten()(x)
x = tf.keras.layers.Dense(128, activation='relu')(x)
outputs = tf.keras.layers.Dense(1, activation='sigmoid')(x)

model = tf.keras.models.Model(inputs=inputs, outputs=outputs)
```

keras_functional_api.py hosted with ❤ by GitHub view raw

# Actor model vs Task model

So far, we’ve described distributing work in a Task model

- Sequential abstraction, runtime runs the task

Actor model: specify what each worker should do with explicit message passing

- Cache coherence lab

# Optional Reading

https://www.youtube.com/live/qGHUuO8Uhc0